

Database를 활용한 헬스케어 데이터 분석

3일차: SQL 실무 + MySQL 실습

목차

- 2일차 복습
- **GROUP BY / HAVING / LIMIT / Aggregation**
- SQL 실무
- Python으로 SQL 실행하기 (PyMysql)

2일차 복습

INNER JOIN

```
1 SELECT * FROM admissions
2 INNER JOIN patients
3 ON admissions.patient_id = patients.patient_id;
```

patient_id	admission_date	discharge_date	diagnosis	attending_doctor_id	patient_id	first_name	last_name	gender	birth_date	city
1	2018-11-06	2018-11-08	Ovarian Dermoid-Cyct	21	1	Donald	Waterfield	M	1963-02-12	Barrie
1	2018-09-20	2018-09-20	Ineffective Breathin Pattern R/T Fluid Accumulatio	24	1	Donald	Waterfield	M	1963-02-12	Barrie
3	2019-01-24	2019-01-29	Cardiac Arrest	2	3	Jiji	Sharma	M	1957-09-05	Hamilton
3	2018-10-21	2018-10-27	Congestive Heart Failure	8	3	Jiji	Sharma	M	1957-09-05	Hamilton

LEFT JOIN

```
1 SELECT patients.first_name, patients.last_name, province_names.province_name FROM patients
2 LEFT JOIN province_names ON province_names.province_id = patients.province_id;
```

first_name	last_name	province_name
Donald	Waterfield	Ontario
Mickey	Baasha	Ontario
Jiji	Sharma	Ontario
Blair	Diaz	Ontario
Charles	Wolfe	Ontario

JOIN + WHERE

```
SELECT A.col1, A.col2, B.col3 FROM table_A
```

```
INNER/LEFT JOIN table_B ON A.key = B.key
```

WHERE 조건

- ON: 테이블 연결 조건
- WHERE: 연결 후 필터링 조건

=> 순서가 어긋나면 데이터 누락이 나올 수도 있음

특히 WHERE안에 JOIN 조건을 사용하면 데이터가 잘리는 경우가 많음

(추가) ALIAS 별칭 / DISTINCT

- Alias: 별칭 -> 짧게 쓰기 가능

SELECT column명 AS 별칭 FROM table_name ;	# Column 별칭
SELECT ... FROM table_name AS 별칭 ;	# Table 별칭

- DISTINCT: 중복 제거

```
SELECT DISTINCT column명 FROM table_name;
```

GROUP BY

```
SELECT * FROM table_A  
GROUP BY group_column;
```

- “데이터를 묶는 기준”
- 예) 성별 / 도시 / 의사별 / 날짜별 등
- GROUP BY는 SELECT 안에 있는 비집계 Column과 일치해야 함

```
1 SELECT * FROM admissions  
2 GROUP BY patient_id;
```

patient_id	admission_date	discharge_date	diagnosis	attending_doctor_id
1	2018-09-20	2018-09-20	Ineffective Breathin Pattern R/T Fluid Accumulatio	24
3	2018-10-21	2018-10-27	Congestive Heart Failure	8
6	2018-06-13	2018-06-15	Asthma Exacerbation	3
7	2018-06-24	2018-07-03	Cancer	8
8	2018-09-18	2018-09-21	Amigima	6

HAVING

```
SELECT * FROM table_A
GROUP BY group_column
HAVING 조건;
```

- GROUP BY 이후 사용 가능 => 그 외 것들과 조합해서 사용 불가
- GROUP BY 결과를 다시 필터링

```
1 SELECT * FROM admissions
2 GROUP BY patient_id
3 HAVING patient_id % 2 == 1;
```

patient_id	admission_date	discharge_date	diagnosis	attending_doctor_id
1	2018-09-20	2018-09-20	Ineffective Breathin Pattern R/T Fluid Accumulatio	24
3	2018-10-21	2018-10-27	Congestive Heart Failure	8
7	2018-06-24	2018-07-03	Cancer	8
9	2018-12-31	2018-12-31	Ruptured Appendicitis	19

LIMIT

```
1 SELECT * FROM patients
2 LIMIT 3;
```

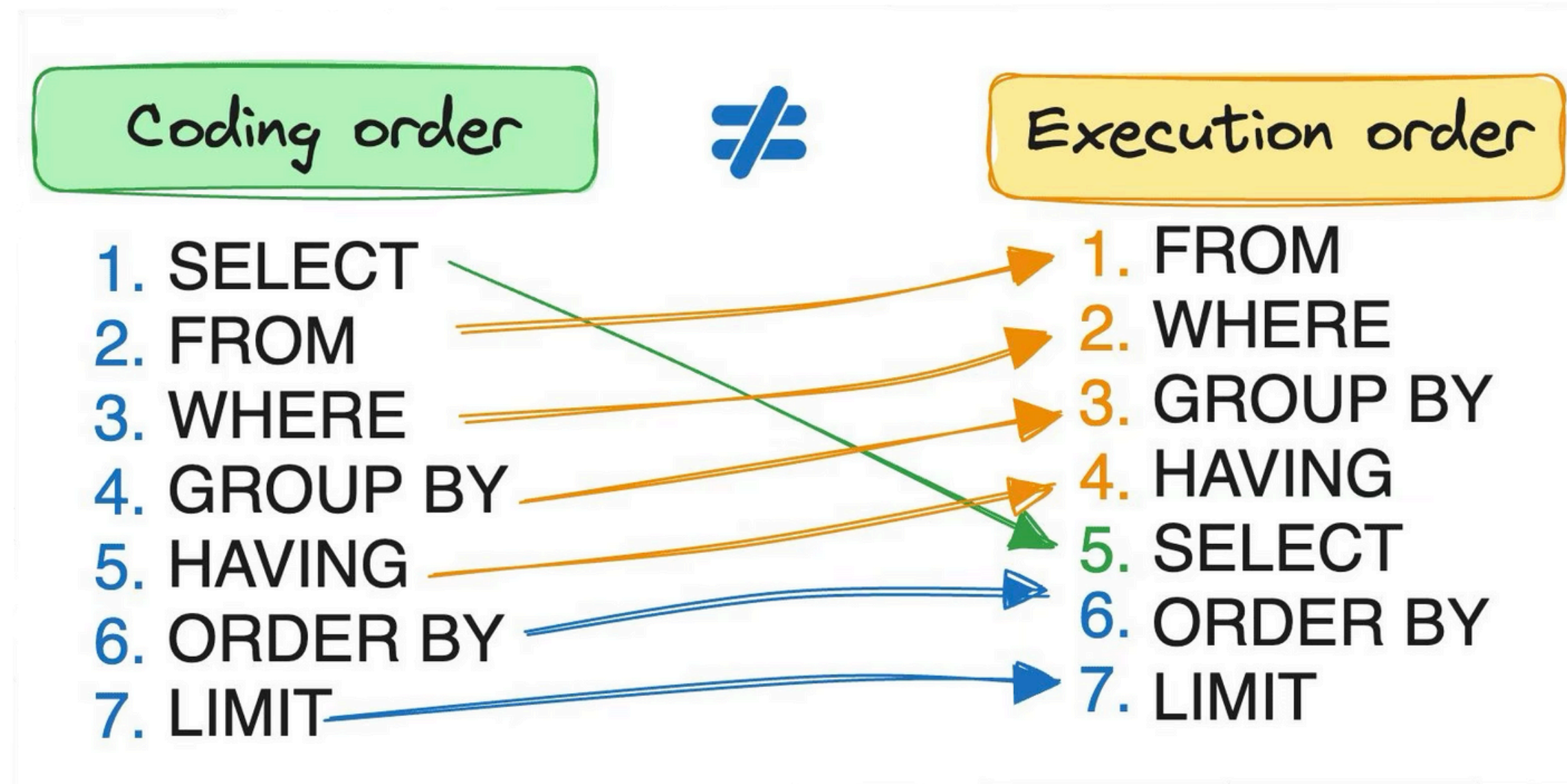
patient_id	first_name	last_name	gender	birth_date	city	province_id	allergies	height	weight
1	Donald	Waterfield	M	1963-02-12	Barrie	ON	NULL	156	65
2	Mickey	Baasha	M	1981-05-28	Dundas	ON	Sulfa	185	76
3	Jiji	Sharma	M	1957-09-05	Hamilton	ON	Penicillin	194	106

```
1 SELECT * FROM patients
2 LIMIT 10, 3;
```

patient_id	first_name	last_name	gender	birth_date	city	province_id	allergies	height	weight
11	Hank	Spencer	M	1969-08-10	Peterborough	ON	NULL	158	74
12	Sara	di Marco	F	1949-04-29	Hamilton	ON	NULL	145	46
13	Daphne	Seabright	F	1954-11-18	Ancaster	ON	Codeine	146	77

**GROUP BY / HAVING /
LIMIT / Aggregation**

SQL 실행 순서



<https://ai.atsit.in/posts/5176260605/>

[실습 1] SQL 실행 순서

- patients와 province_names 테이블을 INNER JOIN하여 province_name 기준으로 GROUP BY를 하세요.

그 후, province_name이 'N'으로 시작하는 지역들만 HAVING으로 필터링하고 province_name 기준 오름차순 정렬 후 상위 3개만 LIMIT하세요.

<https://www.sql-practice.com/>

Aggregation 집계함수

COUNT()	개수
SUM()	합
AVG()	평균
MIN(), MAX()	최소값 / 최대값
STDDEV()	표준편차 (MySQL 8이상)

- 집계 함수에서는 NULL을 자동으로 제외
 - COUNT(column명): NULL 제외
 - COUNT(*): 전체 행 수

Aggregation 집계함수

```
1 SELECT
2 COUNT(patient_id), SUM(height), AVG(weight), MIN(height), MAX(weight)
3 FROM patients;
```

count(patient_id)	SUM(height)	AVG(weight)	MIN(height)	max(weight)
4530	714747	76.77682119205298	25	142

[실습 2] Aggregation 집계함수

- 지역별 평균 키를 구하세요. 그리고 평균 키가 155cm 이상인 지역만 HAVING으로 필터링하세요. 결과는 평균 키 기준 내림차순으로 정렬하고, 상위 3개만 LIMIT하세요.
 - 힌트
 - **patients + province_names INNER JOIN**
 - **province_name 기준 GROUP BY**
 - **AVG(height) 사용**

SQL 실무

(숙제) 로컬 DB 세팅하기

- Windows: 링크
- Mac: 링크1, 링크2

안내에 따라서 MySQL (8.3.0), MySQL Workbench를 설치해주세요.

SQL 기본 작업

- Create -> INSERT
- Read -> SELECT
- Update -> UPDATE
- Delete -> DELETE

지금까지는 데이터 조회만 했다면, 지금부터는 데이터를 직접 핸들링 해보자!!

데이터베이스 생성 & 삭제

생성 및 사용

```
CREATE DATABASE
```

```
db_name; USE db_name;
```

삭제

```
DROP DATABASE db_name;
```

- Workbench에는 여러 개의 DB가 있을 수 있음
- USE로 작업 대상 DB를 선택해야 함

테이블 생성 & 삭제

생성

```
CREATE TABLE table_name  
    ( column1 datatype constraints,  
      column2 datatype constraints,  
      ...  
);
```

```
DROP TABLE table_name;
```

```
CREATE TABLE patient_info (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(50) NOT NULL,  
    gender CHAR(1),  
    birth_date DATE,  
    city VARCHAR(50) DEFAULT 'Unknown'  
);
```


테이블 관계 설정

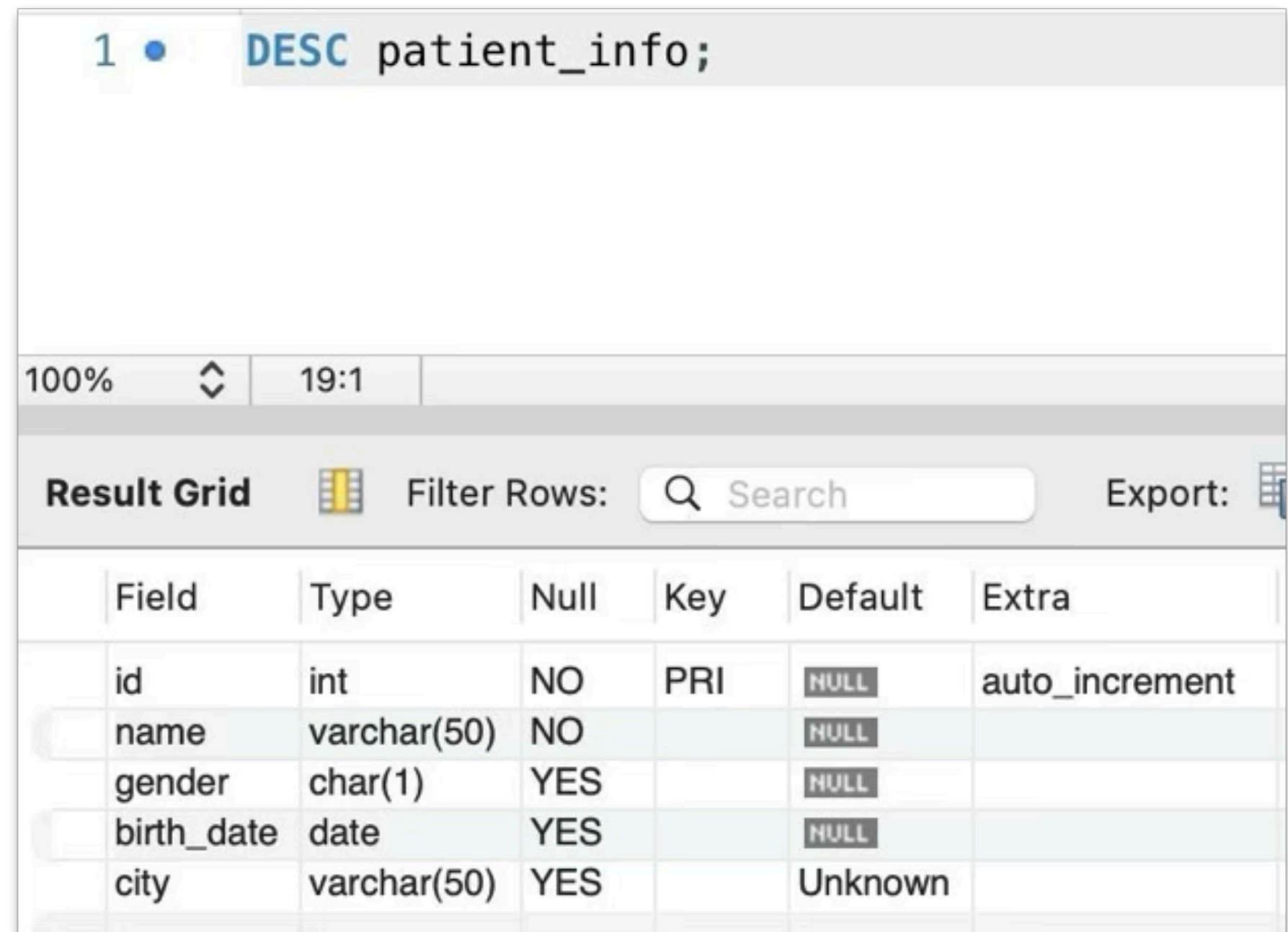
```
CREATE TABLE labs (  
  lab_id INT AUTO_INCREMENT PRIMARY KEY,  
  patient_id INT,  
  lab_name VARCHAR(100),  
  result_value DECIMAL(10,4),  
  result_date DATE,  
  CONSTRAINT fk_labs_patient  
    FOREIGN KEY (patient_id) REFERENCES patients(patient_id)  
);
```

```
CREATE TABLE medications (  
  med_id INT AUTO_INCREMENT PRIMARY KEY,  
  patient_id INT,  
  drug_name VARCHAR(50),  
  dosage DECIMAL(6,2),  
  med_date DATE,  
  CONSTRAINT fk_medications_patient  
    FOREIGN KEY (patient_id) REFERENCES patients(patient_id)  
);
```

DESCRIBE: 스키마 요약본 읽기

DESCRIBE table_name;

- 약자:
DESC



The screenshot shows a database client interface. At the top, a query editor displays the command `DESC patient_info;`. Below the editor, a toolbar shows '100%' zoom, a refresh icon, '19:1' line numbers, and a 'Result Grid' button. To the right of the 'Result Grid' button are 'Filter Rows:' and 'Export:' options. The main area displays a table with the following structure:

	Field	Type	Null	Key	Default	Extra
	id	int	NO	PRI	NULL	auto_increment
	name	varchar(50)	NO		NULL	
	gender	char(1)	YES		NULL	
	birth_date	date	YES		NULL	
	city	varchar(50)	YES		Unknown	

SHOW

테이블 목록 확인 SHOW TABLES;

테이블 생성 SQL 전체 확인
SHOW CREATE TABLE
table_name;

1	SHOW TABLES;
100%	13:1
Result Grid  Filter Rows:	
Tables_in_myt...	
health_score	
patient_info	

3	SHOW CREATE TABLE patient_info;
0%	13:1
Result Grid  Filter Rows: Search Export: 	
Table	Create Table
patient_info	CREATE TABLE `patient_info` (`id` int NOT NULL AUTO_INCREMENT, `name`...

[실습 3] CREATE TABLE

- 다음 구조의 DB 및 Table 3개를 생성해보세요
 - **DB: health_ai**
 - **Table: patients, diagnoses, visits**

[실습 3] CREATE TABLE

- 다음 구조의 DB 및 Table 3개를 생성해보세요
 - patients 테이블

Column	타입	제약	설명
id	INT	PK, AUTO_INCREMENT	환자 식별 기본키
name	VARCHAR(50)	NOT NULL	환자 이름
gender	ENUM('M', 'F')		성별 (남/여)
birth_date	DATE		생년월일

[실습 3] CREATE TABLE

- 다음 구조의 DB 및 Table 3개를 생성해보세요
 - diagnoses 테이블

Column	타입	제약	설명
diag_id	INT	PK, AUTO_INCREMENT	진단 ID
patient_id	INT	FK -> patients(patient_id)	어떤 환자에게 내린 진단 인지 연결
diagnosis_name	VARCHAR(50)		진단명
diag_date	DATE		진단 날짜

[실습 3] CREATE TABLE

- 다음 구조의 DB 및 Table 3개를 생성해보세요
 - visits 테이블

Column	타입	제약	설명
visit_id	INT	PK, AUTO_INCREMENT	방문 기록 ID
patient_id	INT	FK -> patients(patient_id)	방문한 환자
visit_date	DATETIME		방문한 날짜/시간
doctor	VARCHAR(50)		진료한 의사 이름

ALTER TABLE

- Column 핸들링
- 실행 시간이 길 -> 운영환경에서는 자주 사용 못함

ALTER TABLE **table_name** # Column 추가
ADD COLUMN **new_column_name datatype;**

ALTER TABLE **table_name** # Column 삭제
DROP COLUMN
column_name;

ALTER TABLE **table_name** # Column 수정
MODIFY **column_name datatype;**

ALTER TABLE

```
-- PRIMARY KEY 추가
ALTER TABLE patient_info
ADD PRIMARY KEY (id);

-- PRIMARY KEY 삭제
ALTER TABLE patient_info
DROP PRIMARY KEY;

-- FOREIGN KEY 추가
ALTER TABLE appointment
ADD CONSTRAINT fk_patient
    FOREIGN KEY (patient_id)
    REFERENCES patient_info(id);

-- FOREIGN KEY 삭제
ALTER TABLE appointment
DROP FOREIGN KEY fk_patient;
```

```
-- 컬럼 추가
ALTER TABLE patient_info
ADD phone VARCHAR(20);

-- 컬럼 삭제
ALTER TABLE patient_info
DROP COLUMN phone;

-- 컬럼 타입 변경
ALTER TABLE patient_info
MODIFY name VARCHAR(100);

-- 컬럼 이름 변경
ALTER TABLE patient_info
CHANGE gender sex CHAR(1);

-- 기본값 변경
ALTER TABLE patient_info
ALTER city SET DEFAULT 'N/A';

-- 테이블 이름 변경
ALTER TABLE patient_info
RENAME TO patient_data;
```

[실습 4] ALTER TABLE

실습 3번의 테이블을 다음의 조건을 만족하도록 수정하세요.

- patients 테이블: 나이(age, INT) column을 추가하세요
- visits 테이블: doctor column을 hospital VARCHAR(100)으로 변경하세요.

INSERT

```
INSERT INTO table_name (col1, col2, col3)  
VALUES ('값1', '값2', '값3');
```

```
INSERT INTO patient_info (name, gender, birth_date, city)  
VALUES ('Emily Park', 'F', '1990-07-21', 'Seoul');
```


UPDATE

```
UPDATE table_name  
SET column1 = value1, column2 = value2  
WHERE 조건;
```

[주의] WHERE를 빼면 전체 데이터가 변경됨!!

```
UPDATE patient_info  
SET city = 'Toronto'  
WHERE id = 1;
```

DELETE

DELETE FROM **table_name**
WHERE 조건;

- 실무에서는 DELETE 대신 soft delete 사용
- 특히 병원 시스템에서는 DELETE 거의 금지

```
DELETE FROM patient_info  
WHERE id = 3;
```

[실습 5] MySQL 간단 프로젝트

실습 4번의 테이블에 데이터를 삽입 후 질문에 대한 쿼리를 작성하세요.

patient_id	name	age	gender	birth_date
1	Alice	34	F	1991. 4. 2.
2	Brian	52	M	1973. 11. 2.
3	Clara	29	F	1996. 8. 30.
4	Daniel	47	M	1980. 2. 17.
5	Eva	60	F	1965. 9. 25.

patients 테이블

[실습 5] MySQL 간단 프로젝트

실습 4번의 테이블에 데이터를 삽입 후 질문에 대한 쿼리를 작성하세요.

diagnosis_id	patient_id	diagnosis_name	diag_date
101	1	Hypertension	2024. 12. 20.
102	2	Diabetes	2025. 1. 5.
103	2	Hypertension	2025. 2. 1.
104	4	Arrhythmia	2024. 11. 14.
105	5	Hypertension	2025. 1. 22.

diagnosis 테이블

[실습 5] MySQL 간단 프로젝트

실습 4번의 테이블에 데이터를 삽입 후 질문에 대한 쿼리를 작성하세요.

visit_id	patient_id	visit_date	hospital
1001	1	2025-01-03 10:00:00	Seoul Clinic
1002	1	2025-02-10 13:10:00	Seoul Clinic
1003	3	2025-01-25 15:30:00	Busan Center
1004	4	2025-03-20 09:20:00	Incheon Gen.
1005	5	2025-02-14 11:45:00	Daegu Hosp.

visits 테이블

[실습 5] MySQL 간단 프로젝트

실습 4번의 테이블에 데이터를 삽입 후 질문에 대한 쿼리를 작성하세요.

1. **고혈압 환자 찾기:** 진단명이 Hypertension인 환자 찾기

2. **대사 질환 위험군 찾기:**

a. Diabetes 당뇨

b. Hypertension 고혈압

3. **최근에 방문한 고위험 환자 TOP 3 찾기:**

a. Diabetes 당뇨

b. Hypertension 고혈압

c. 중 하나라도 있으면 고위험 환자로 분류

d. 최근: 방문 날짜 기준

Python으로 SQL 실행하기

PyMySQL

- 파이썬에서 MySQL 데이터베이스와 쉽게 상호작용할 수 있게 해주는 라이브러리
- `pip install pymysql -> import pymysql`

DB 연결: connect()

```
import pymysql

conn = pymysql.connect(
    host='localhost',
    user='root',
    password='비밀번호',
    db='training_db',
    charset='utf8mb4',
    cursorclass=pymysql.cursors.DictCursor
)

cursor = conn.cursor()
```

SQL 실행한다고 신호주기: cursor()

```
cur = conn.cursor()
```



```
with conn.cursor() as cur:  
    ...
```

SQL 실행하기: execute()

```
cur.execute("SELECT * FROM patient_info;")  
row = cur.fetchone()  
rows = cur.fetchall()
```

함수	의미	반환
fetchone()	결과 1행만	dict 1개
fetchall()	전체 결과	list[dict]
fetchmany(n)	n개 행만	list[dict]

변경사항 적용/되돌리기: commit() / rollback()

```
try:
    cur.execute(
        "UPDATE patient_info SET city=%s WHERE id=%s",
        ('Busan', 2)
    )
    conn.commit()
    print("정상 업데이트 → commit 완료")

except Exception as e:
    conn.rollback()
    print("에러 발생! 롤백 실행")
```

CREATE, INSERT, UPDATE, DELETE

```
# CREATE TABLE
cur.execute("""
CREATE TABLE patient_info (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    gender CHAR(1),
    birth_date DATE,
    city VARCHAR(50) DEFAULT 'Unknown'
)
""")
conn.commit()

# INSERT
cur.execute(
    "INSERT INTO patient_info (name, gender, birth_date, city) VALUES (%s, %s, %s, %s)",
    ('David Kim', 'M', '1988-02-19', 'Incheon')
)
conn.commit()
```


CREATE, INSERT, UPDATE, DELETE

```
# UPDATE
cur.execute(
    "UPDATE patient_info SET city=%s WHERE id=%s",
    ('Seoul', 1)
)
conn.commit()

# DELETE
cur.execute(
    "DELETE FROM patient_info WHERE id=%s",
    (1,)
)
conn.commit()
```

JOIN + ORDER BY + LIMIT

```
cur.execute("""  
SELECT d.first_name, COUNT(*) AS visits  
FROM appointments a  
JOIN doctors d ON a.doctor_id = d.doctor_id  
GROUP BY d.doctor_id  
ORDER BY visits DESC  
LIMIT 3  
""")  
cur.fetchall()
```

[실습 6] CSV -> Pandas -> MySQL (PyMySQL)

실시간으로 진행합니다.

1. CSV 파일을 pandas로 읽기
2. 데이터 정제
3. DataFrame -> Python dict 변환
4. MySQL Table 생성
5. CSV 데이터를 실제 DB테이블로 옮기기: INSERT (한줄 / 여러 줄)
6. DB 확인하기

QnA